CSC3100: List

Tianshu Yu

2024 Fall

Overview

- Introduction to linked list
- Variants of linked list
- Basic operations
- Examples and implementation

List

- Definition in Wikipedia:
  - A list is an abstract data type (ADT) that represents a finite number of ordered values, where the same value may occur more than once

- A list: $a_1, a_2, a_3, \ldots, a_N$
  - We say that the size of this list is N
  - For any list except the empty list, we say:
    - The first element of the list is a_1, and the last element is a_N
    - a_{i+1} follows (succeeds) a_i ($i < N$)
    - a_{i-1} precedes a_i ($i > 1$)
    - The predecessor of a_1, or the successor of a_N are not defined

Array vs. Linked Lists

- Arrays are suitable for:
  - Inserting/deleting an element at the end.
  - Randomly accessing any element.
  - Searching the list for a particular value.
- Linked lists are suitable for:
  - Inserting an element.
  - Deleting an element.
  - Applications where sequential access is required.
  - In situations where the number of elements cannot be predicted beforehand.

Advantages

- Dynamic data structure:
  - The size of a linked list is not fixed as it can vary arbitrarily
- Insertion and deletion are easier:
  - In linked lists, insertion and deletion are easier than those on arrays, since the elements of an array are stored in a consecutive location, while the elements of a linked list are stored in a random location
  - If we want to insert or delete the element in an array, then we need to shift the elements for creating the space, while in a linked list, we do not have to shift the elements
- Memory efficient:
  - Its memory consumption is efficient as the size of the linked list can grow or shrink according to our requirements

Disadvantages

- Memory usage:
  - The node in a linked list occupies more memory than array as each node occupies two types of variables, i.e., one is a simple variable, and the other is a pointer variable that occupies 4 bytes in memory
- Traversal:
  - In an array, we can randomly access the element by index, while in a linked list, the traversal is not easy, i.e., if we want to access the element in a linked list, we cannot access the element randomly
- Reverse traversing:
  - In a linked list, backtracking or reverse traversing is difficult
  - In a doubly linked list, it is easier but requires more memory to store the back pointer

Types of linked list

- We focus on linked list
  - Singly linked list
  - Doubly linked list
  - Circular linked list

Lists [edit]

- Doubly linked list
- Array list
- Linked list
- Association list
- Self-organizing list
- Skip list
- Unrolled linked list
- VList
- Conc-tree list
- Xor linked list
- Zipper
- Doubly connected edge list also known as half-edge
- Difference list
- Free list

Linked List



- Consists of a series of nodes (Node class)
- A singly linked list, each node is composed of data and a reference
- Not necessarily adjacent in memory
- Flexible on element insertion and deletion
- May contain a 'head' node or not.
  - why add a 'head'?

Note:
"*head*" here is a node that does NOT store data

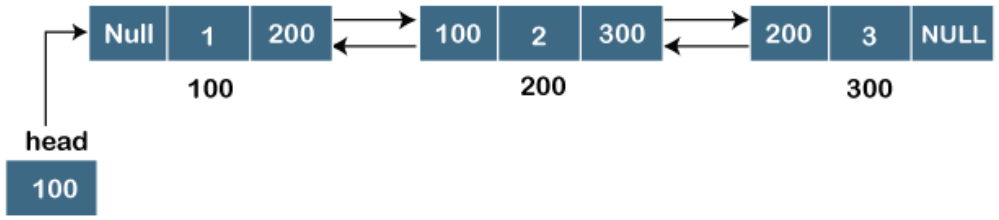
Applications of singly linked list

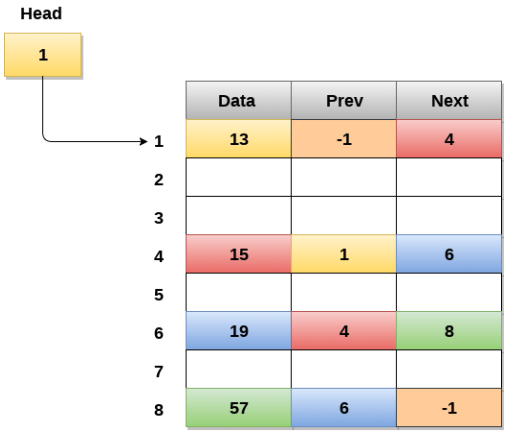
- It is used to implement stacks and queues, which are fundamental data structures
- To prevent the collision between the data in the hash map, we use a singly linked list
- A casual notepad uses a singly linked list to perform undo/redo functions
- …
- More detailed examples will be given in next lecture

Doubly linked list

- A doubly linked list has three parts in a node
  - A data part
  - A pointer to its previous node
  - A pointer to its next node

```
class Node {
     int element;
     Node next;
     Node prev;
}
```



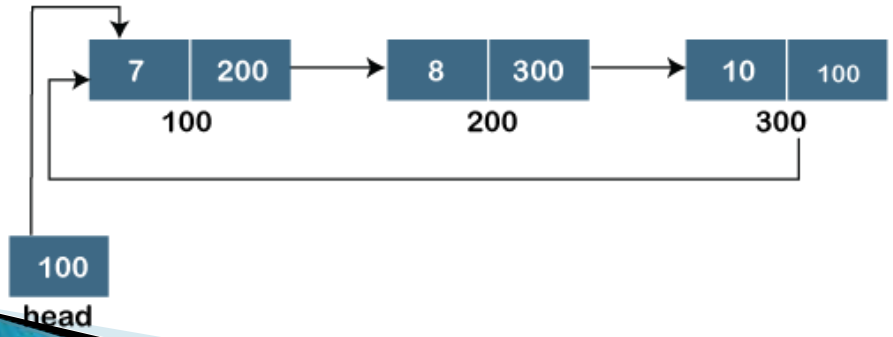


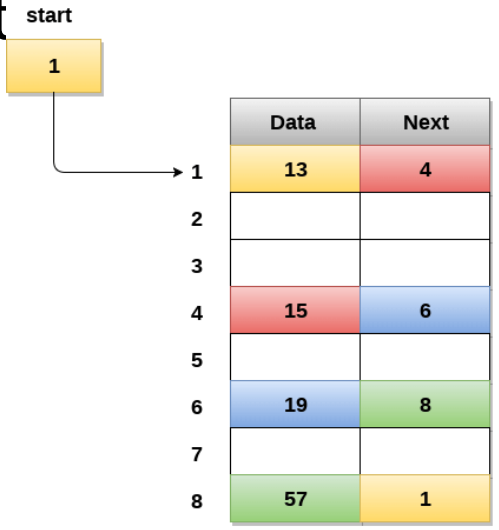
Memory Representation of a Doubly linked list

Circular linked list

- It is a variation of a singly linked list

- Singly linked list vs a circular linked list
  - In a singly linked list, the last node does not point to any node
  - In a circular linked list, the last node links to the first node
  - The circular linked list has no st[illegible] so we can traverse in any direction





Memory Representation of a circular linked list

Generic type in Java

- **parametric polymorphism**: Ability for a function or type to be written in such a way that it handles values identically without depending on knowledge of their types.
- Motivation: Parametric polymorphism allows you to write flexible, general code without sacrificing type safety.
  - Most commonly used in Java with collections.
  - Also used in reflection (*seen later*).

Type Parameters (Generics)

```
List<Type> name = new ArrayList<Type>();
```

- `java.util.ArrayList` supports generic type. We specify the type of elements it will contain between < and >.
  - We say that the `ArrayList` class accepts a **type parameter**, or that it is a **generic** class.
  - Use of the "raw type" `ArrayList` without <> leads to warnings.

```
List<String> names = new ArrayList<String>();
names.add("Marty Stepp");
names.add("Stuart Reges");
String teacher = names.get(0);        // no cast
Point oops = (Point) names.get(1);    // error
```

The Node Class

```
public class Node<T>
{
   public T nodeValue;     // data held by the node
   public Node<T> next;    // next node in the list

   public Node()
   { nodeValue = null;
     next = null;
   }

   public Node(T item)
   { nodeValue = item;
     next = null;
   }
}
```

List ADT and basic operations

- Some popular operations on List ADT are:
  - printList
  - makeEmpty
  - Find
    - return the position of the first occurrence of a key, e.g., given the list: 34, 12, 52, 16, 12, find(52) returns 3
  - insert
    - insert some key at some position, e.g., insert(X, 3)
  - delete
    - delete some key from some position, e.g., delete(52)
  - next & previous, concatination (optional)

Basic Operations on a List

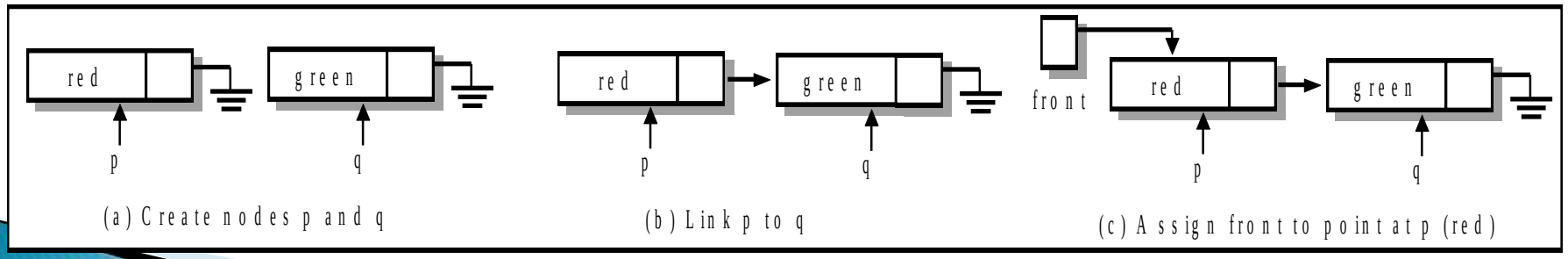
- Creating a list
- Scanning the list
- Inserting an item in the list
- Deleting an item from the list
- Concatenating two lists into one

Creating a Linked List

```
// create two nodes (figure (a)
Node<String> p = new Node<String>("red");
Node<String> q = new Node<String>("green")

// link p to q
p.next = q;                    // figure (b)

// set front to point at the first node
Node<String> front = p;     // figure (c)
```



continued

Scanning a Linked List

- We scan a singly linked list by starting at the front, and then move along the list one Node at a time
  - sequential access (O(n))
  - stop when we reach null
- toString() is an example of a scanning method, which builds a string as it moves along the list.

toString()

```
public static <T> String toString(Node<T> front)
// build a string from the list of the form
//     "[ n1, n2, ..., nx ]"
{
   if (front == null)    // empty list
      return "[]";

   Node<T> curr = front;      // start at the front

   String s = "[" + curr.nodeValue;
   while(curr.next != null) {
      curr = curr.next;       // move along list
      s += ", " + curr.nodeValue;
   }
   s += "]";
   return s;
}
```

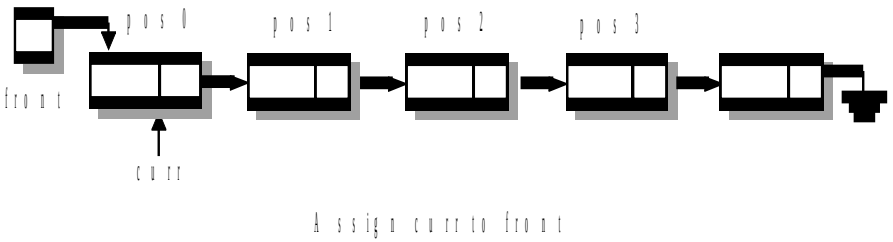
Moving to a List Position

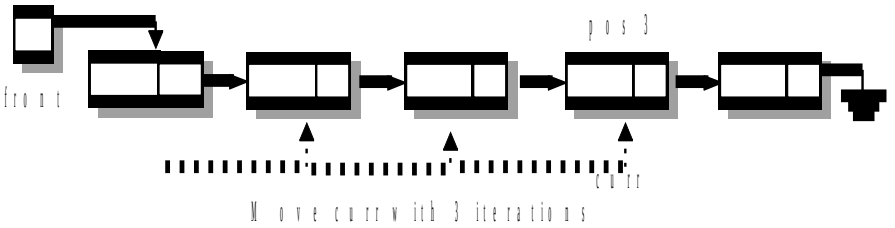
- To move to an element at position i, we need to start at the front and move through the list counting up to i
  - sequential access again (O(n))
- The first element of the list is at position 0.

continued

```
Node<T> curr = front;   // start at the front of list

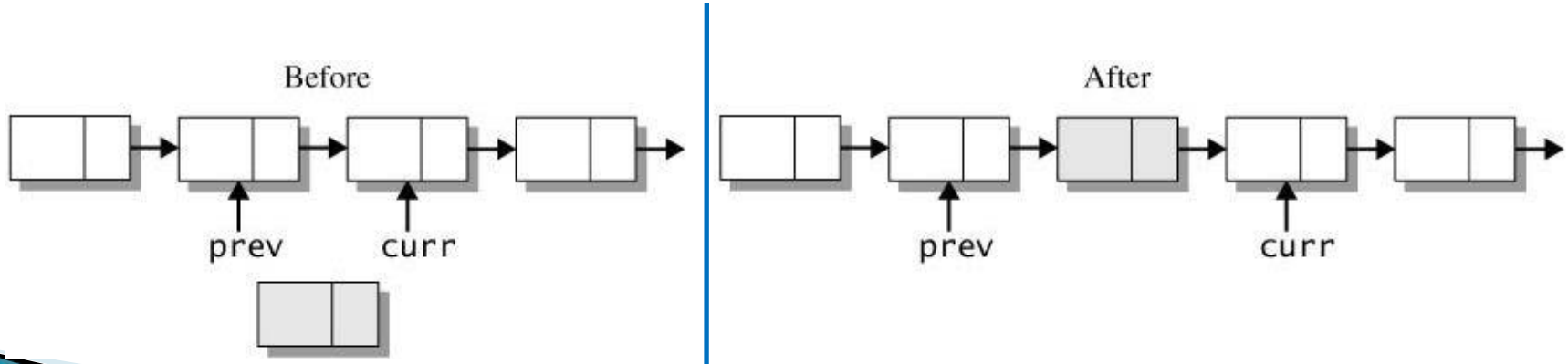
for (int i = 0; i < xPos; i++)
  curr = curr.next;        // move along list
```





Insertion

- To insert a new node before a node referenced by 'curr', the code must have access to the previous node, 'prev', since its link must be changed.

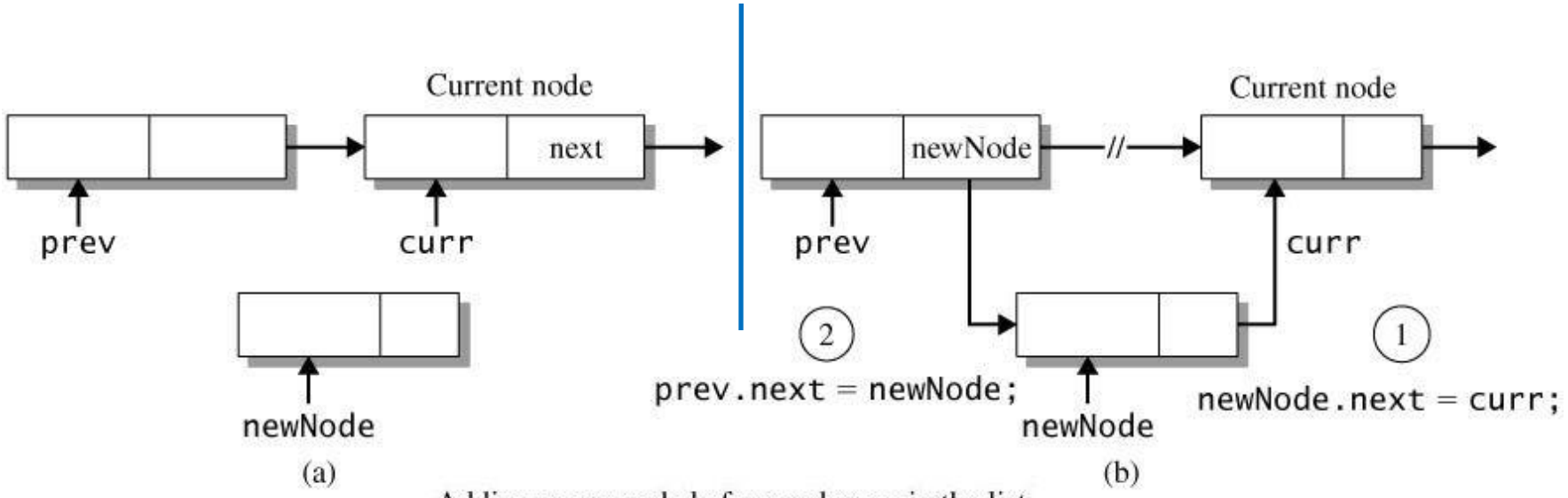


continued

```
Node<T> curr =  ... // set to point to a node
Node<T> prev =  ... // set to point to previous node

Node<T> newNode = new Node<T>(item);  // new node

// update links
newNode.next = curr;        // step 1
prev.next = newNode;        // step 2
```



Adding a new node before node curr in the list.

continued

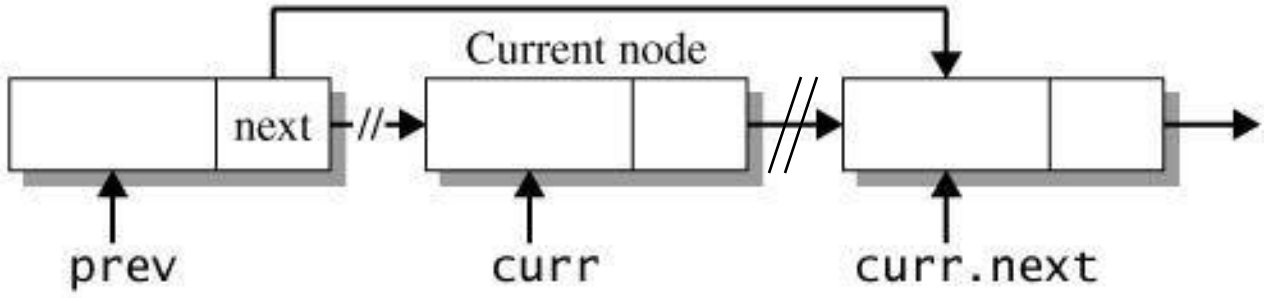
- The insertion is O(1) since only two links need to be changed. But the real cost is the sequential search to find the insertion position, which is O(n).

Deletion

- Deleting a node at position curr requires access to the predecessor node prev.

```
Node<T> curr =  ... // set to point to a node
Node<T> prev =  ... // set to point to previous node

// connect prev to curr.next
prev.next = curr.next;
curr.next = null;
```



Removing a node at position curr in the list.

continued

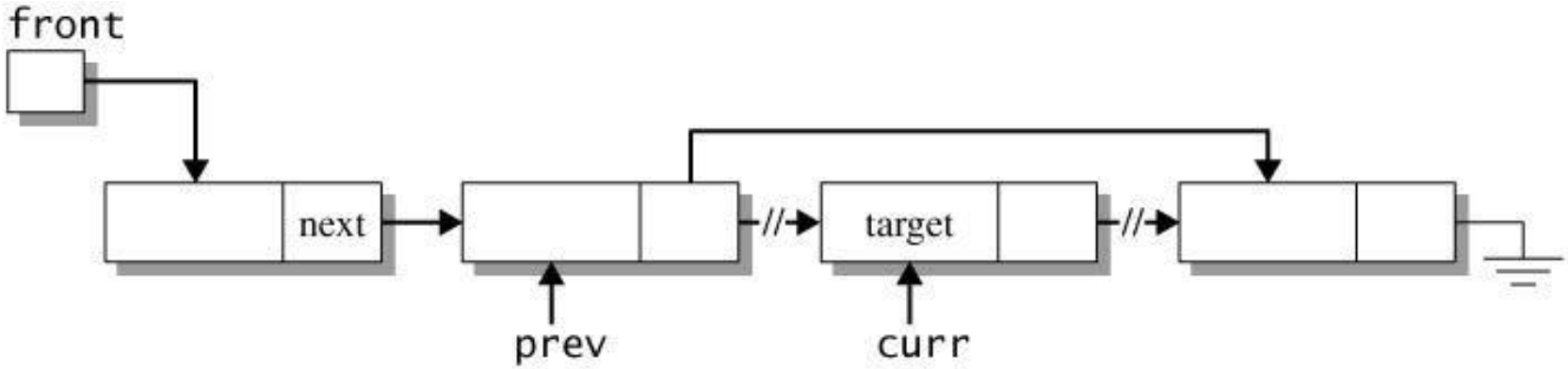
- The deletion is O(1) since only two links need to be changed. But the real cost is the sequential search to find the deletion position, which is O(n)
  - this is shown in the remove() method, which is explained next

Removing a Target Node

- To remove the first node having a specified value, scan the list to find the node.
- Suppose there is no ‘head’ node
- The scan must use two references that move together down the list
  - one reference (curr) points to the current node in the scan
  - the other reference (prev) points to the previous node (why?)

continued

- Once 'curr' finds the node, the code uses 'prev' to unlink 'curr'.



Positions of prev and curr for a deletion.

continued

- At the start, point 'curr' at the front of the list and set 'prev' to null, since the first node does not have a predecessor.

- Move 'curr' and 'prev' down the list until curr.nodeValue matches the target or curr == null.

continued

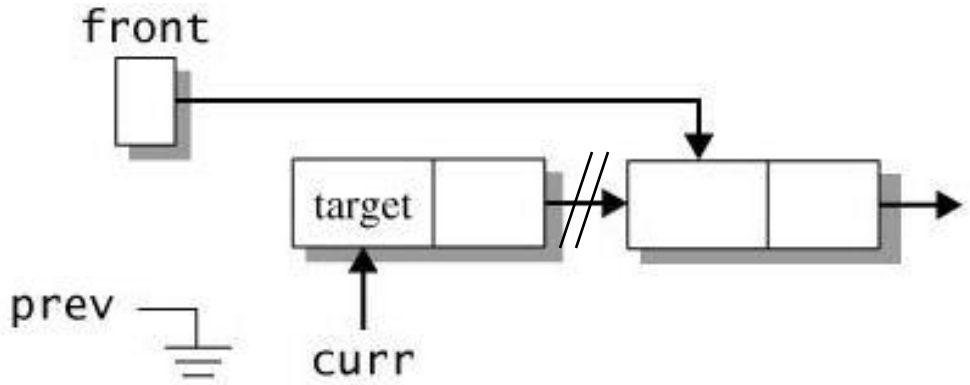
- If the target is found then 'curr' points at the node and 'prev' to the predecessor node.

- But there are *two* possible cases:
  - the target node is the first node, so 'prev' is null
  - the target node is *not* the first node, so 'prev' points to something

continued

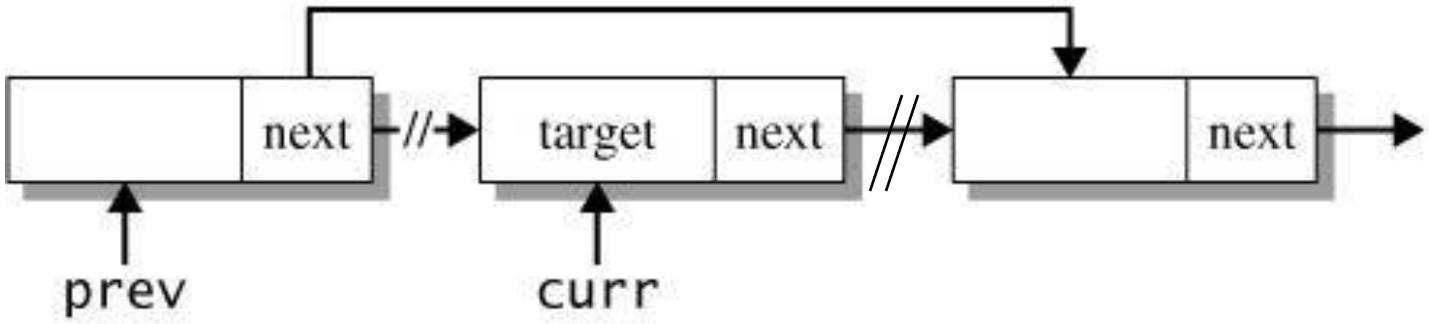
- *Case 1*: 'prev' is null which means that 'curr' points to the first node.
  - so we only have to delete the front of the list

```
front = curr.next;
curr.next = null;
```



- *Case 2*: The match occurs in the middle of the list. Both 'curr' and 'prev' have non-null values. Unlink the current node.

```
prev.next = curr.next;
curr.next = null;
```



continued

- The generic remove() is passed a reference to the front of the list and the target value.
- The method returns the value of 'front', which may have been updated if the first node was deleted.

remove() Method

```
public static <T> Node<T> remove(Node<T> front, T target)
/* Delete the first occurrence of the target in the
   linked list referenced by front; return the
   value of front */
{
   // initialize pointers
   Node<T> curr = front;
   Node<T> prev = null;

   boolean foundItem = false;
     // set to true if we find the target
            :
```

```
    // scan until find item or end of list (O(n))
    while (curr != null && !foundItem) {
       // check for a match
       if (target.equals(curr.nodeValue)) {
          if (prev == null)   // remove first Node (O(1))
            front = front.next;
          else       // erase middle Node (O(1))
            prev.next = curr.next;
          curr.next = null;
          foundItem = true;
       }
       else {  // advance curr and prev
         prev = curr;
         curr = curr.next;
       }
    }
    return front;  // may be updated
}   // end of remove()
```

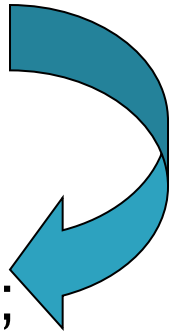
Question

- What will happen if there is a ‘head’ for implementing `remove()`?
- How to swap two elements (at position i and j) in a linked list?

List: Example1

- Single-variable Polynomials

$$F(X) = \sum_{i=0}^{N} A_i X^i$$



By array implementation

```
class Polynomial {
        int coeffArray[MaxDegree + 1];
        int highPower;
}
```

Example1

- Initialize a polynomial

```
void zeroPolynomial(Polynomial poly){
        int j;
        for (j = 0;  j <= MaxDegree; j++)
                poly.coeffArray[j] = 0;
        poly.highPower = 0;
}
```

Example1

- Add two polynomials

```
void addPolynomial(Polynomial poly1, Polynomial poly2,
    Polynomial polySum) {
    zeroPolynomial(polySum);
    polySum.highPower = Math.max(poly1.highPower,
    poly2.highPower);
    for (int i = polySum.highPower; i>=0; i--)
          polySum.coeffArray[i] = poly1.coeffArray [i] +
                poly2.coeffArray[i];
}
```

Example1

- Multiply two polynomials

```
void multPolynomial(Polynomial poly1, Polynomial poly2,
    Polynomial polyProd) {
    zeroPolynomial(polyProd);
    polyProd.highPower = poly1.highPower+ poly2.highPower;
    If (polyProd.highPower > MaxDegree)
          Error("Exceed array size");
    else
          for (int i =0; i<=poly1.highPower; i++)
            for (int j =0; j<=poly2.highPower; j++)
              polyProd.coeffArray[i+j] += poly1.coeffArray [i] *
                                          poly2.coeffArray[j];
}
```

Example1

- Good or bad?
- Consider the following situation

  $P_1(X) = 10\,X^{1000} + 5X^{14} + 1$

  $P_2(X) = 3X^{1990} - 2X^{1492} + 11X + 5$

- Most of the time is spent multiplying zeros

Example1

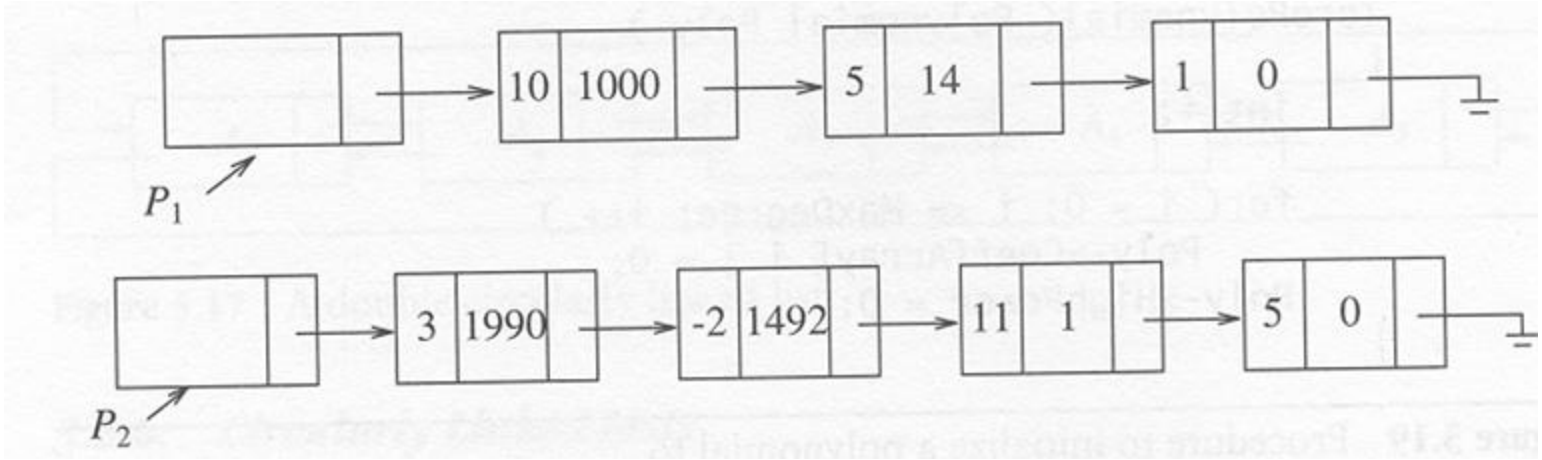
- Multiply two polynomials: better structure

```
class Node {
        int     coefficient;
        int     exponent;
        Node next;
}
```

Example1

$$P_1(X) = 10\,X^{1000} + 5X^{14} + 1$$
$$P_2(X) = 3X^{1990} - 2X^{1492} + 11X + 5$$

- Linked list representation of the previous structure



Implementation exercises

- Suppose we have two polynomials represented by linked lists, with m and n nodes, which are sorted according to their degrees
  - Write the pseudocodes of adding them
    - Hint: recall the merge function in MergeSort; create a new linked list and merge two linked lists where nodes with the same degrees are added together
  - Write the pseudocodes of multiplying them
    - Hint: create m linked lists and then merge them; create a new linked list and merge m linked lists

Example2

- A university with 40,000 students and 2,500 subjects needs to generate 2 reports:
  1. Lists of students for each class
  2. Lists of classes that each student registered.

  Implementation: construct 2D array (40Kx2.5K matrix) = 100M entries

  if each student takes 3 subjects => only 120K entries (~0.1% of 100M) => waste of resources.

Example2

- First solution: triplet representation (minimum space)

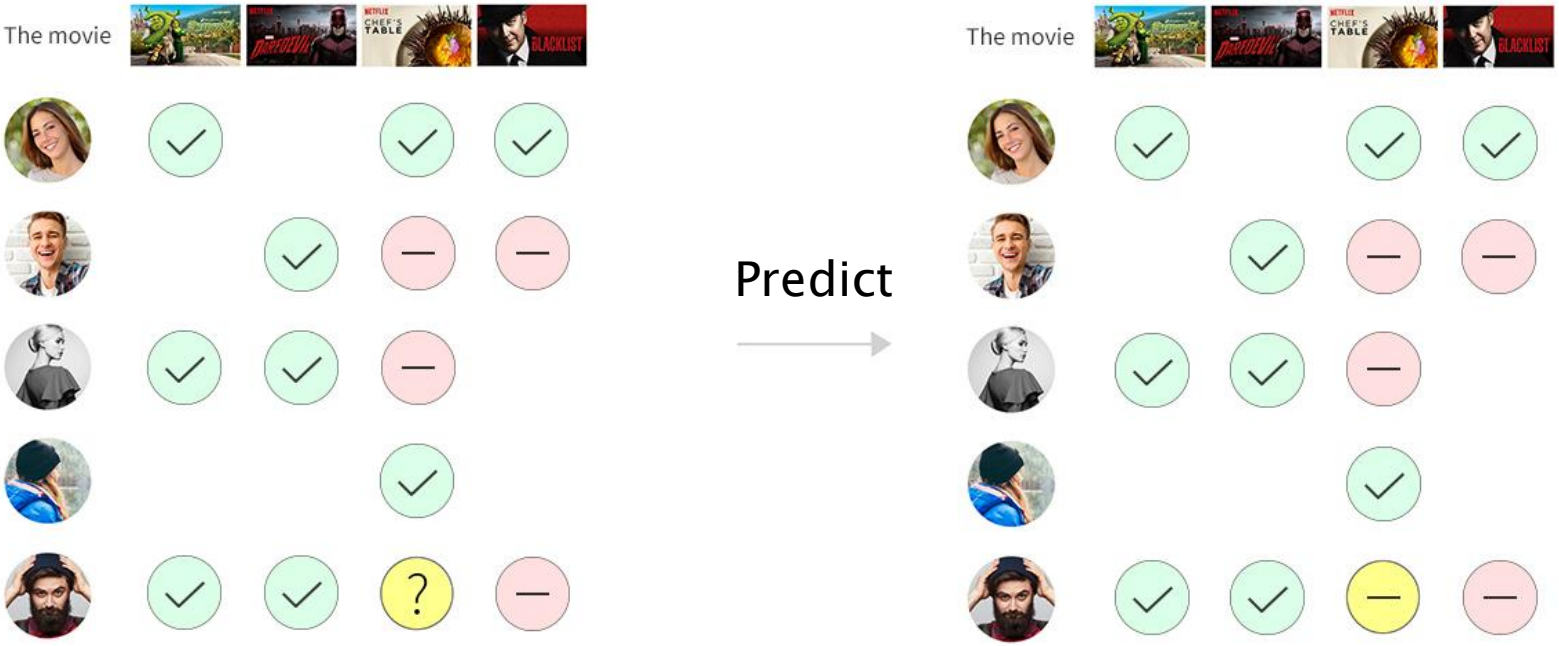
$$\begin{bmatrix} 0 & 0 & 0 & 0 & 9 & 0 \\ 0 & 8 & 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 5 \\ 0 & 0 & 2 & 0 & 0 & 0 \end{bmatrix}$$



Rows	Columns	Values
0	4	9
1	1	8
2	0	4
2	2	2
3	5	5
4	2	2

Example3

- Recommendation problem



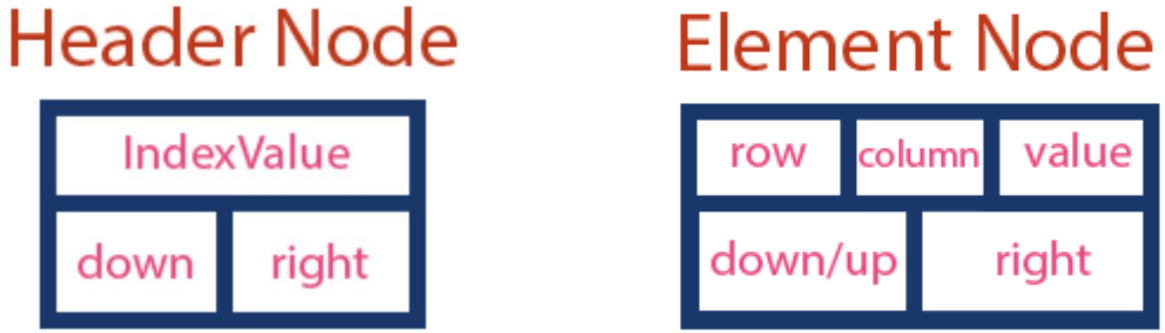
- In a lot of applications, such as music app, taobao, meituan, seems everywhere in social network

Example3

- Suppose we have 10 million people and tens of movies or products
  - 10,000,000*10,000 = 100G items, how to save?
- The huge matrix may involve in many computations, such as multiplications, eigen decomposition, etc.
  - Solution: spare data structure based on basic type

Example 3: sparse matrix

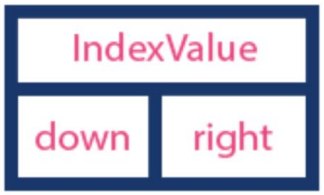
- We represent each column (row) of a sparse matrix as a circularly linked list with a header node.
- Each element node is simultaneously linked into two lists: a row list, and a column list.
- Each head node is belonged to three lists: a row list, a column list, and a header node list.
- Two header lists exist, corresponding to row and column with length num_rows and num_cols respectively.



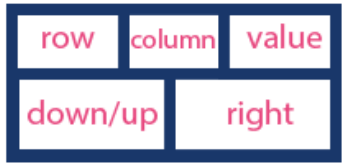
Example3: sparse matrix

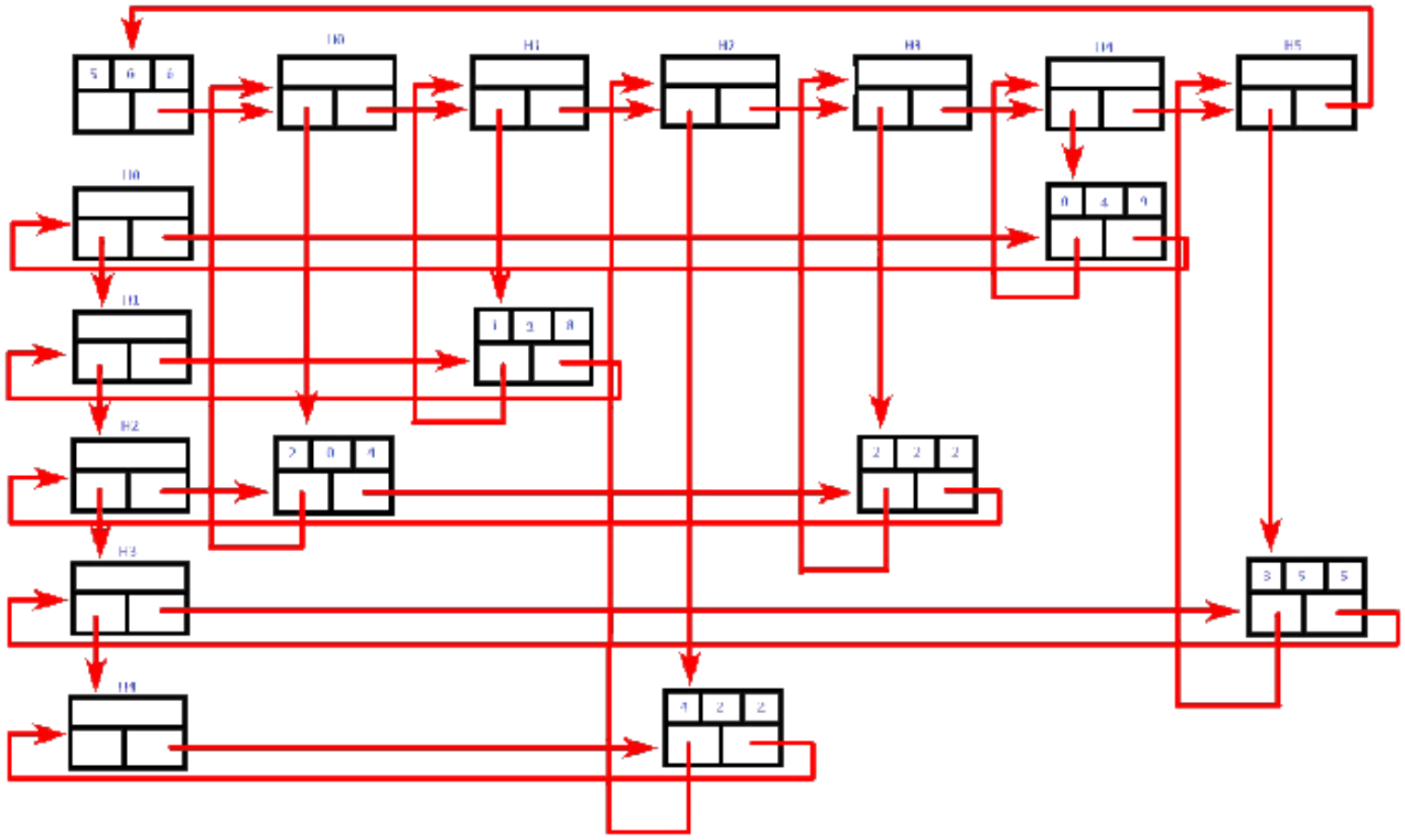
- Second solution: Linked Representation

Header Node



Element Node



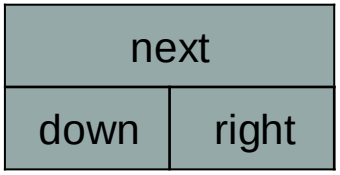


Questions

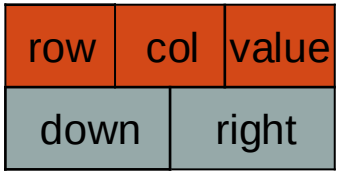
- How can you let a reference be either a header node or an element node?
- How to insert an element at the (i, j)th position?
- How to delete an element at the (i, j)th position?

Example4: another sparse matrix

- We represent each column (row) of a sparse matrix as a circularly linked list with a header node.
- Each element node is simultaneously linked into two lists: a row list, and a column list.
- Each head node is belonged to three lists: a row list, a column list, and a header node list.
- **The header node for row i is also the header node for column i. The number of header nodes is max{ num_rows, num_cols}.**



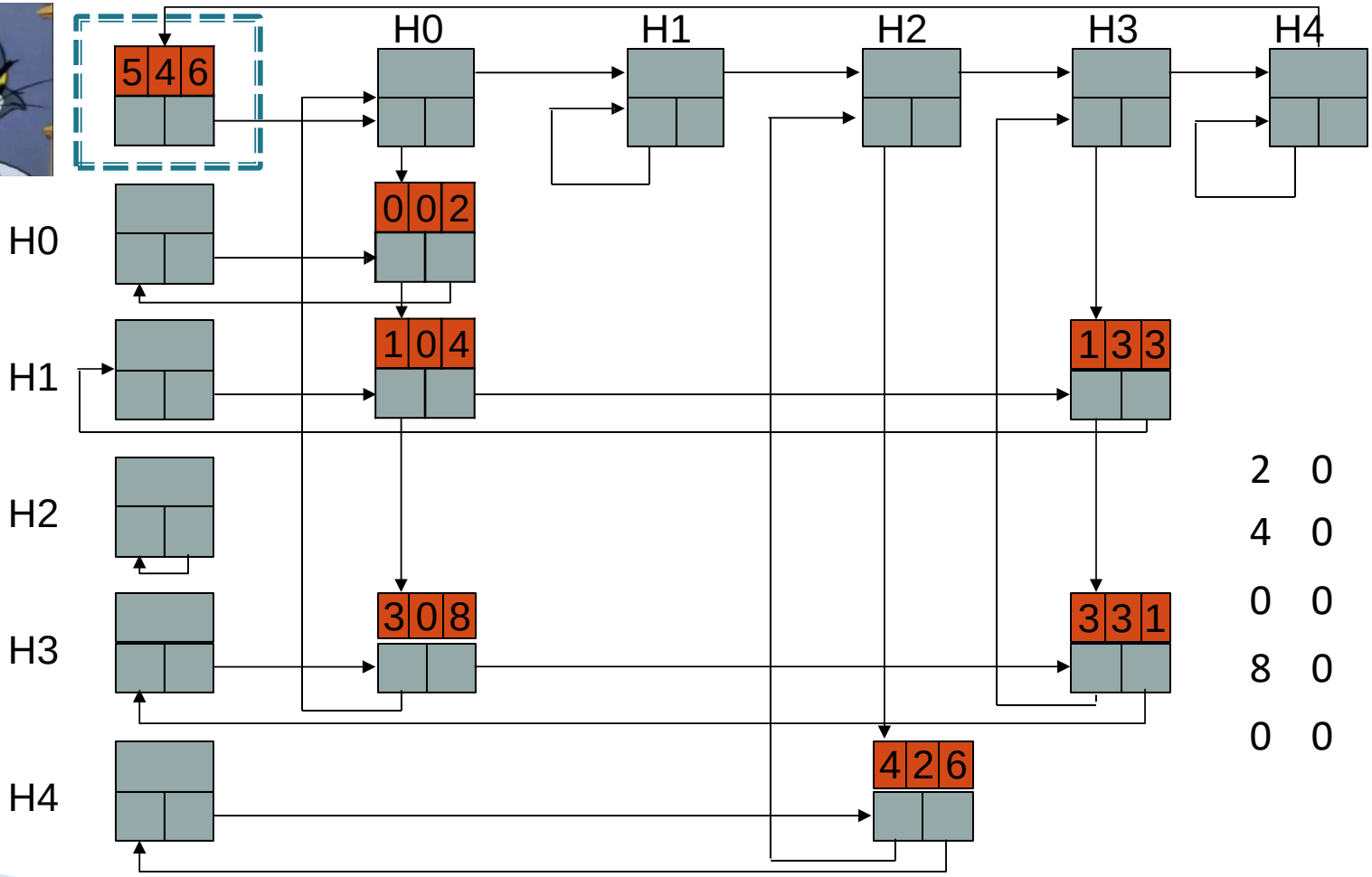
header node



element node

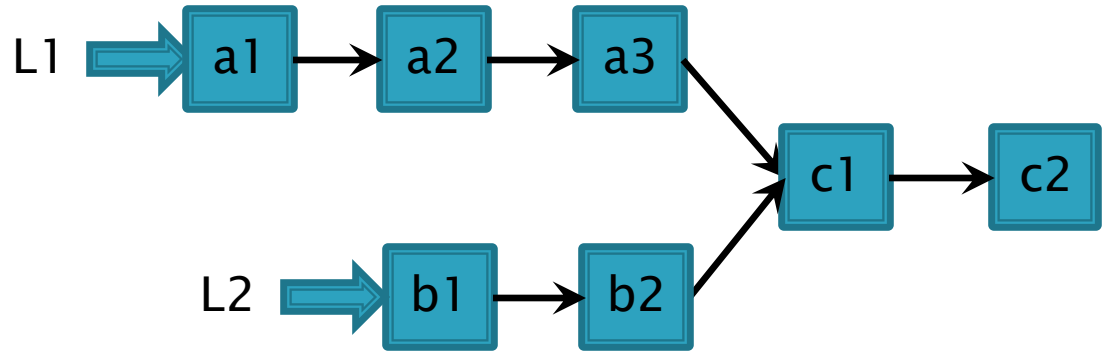
Example : 4 x 5 sparse matrix





2	0	0	0
4	0	0	3
0	0	0	0
8	0	0	1
0	0	6	0

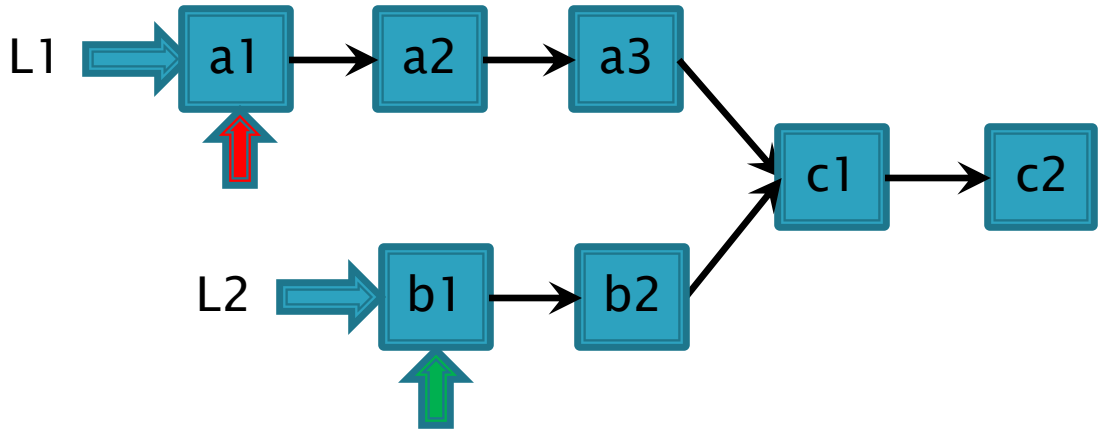
Exercise 1: intersection of two linked lists



Setting: L1 has M elements while L2 has N elements

Goal: find the first node where L1 and L2 intersect

Exercise 1: intersection of two linked lists



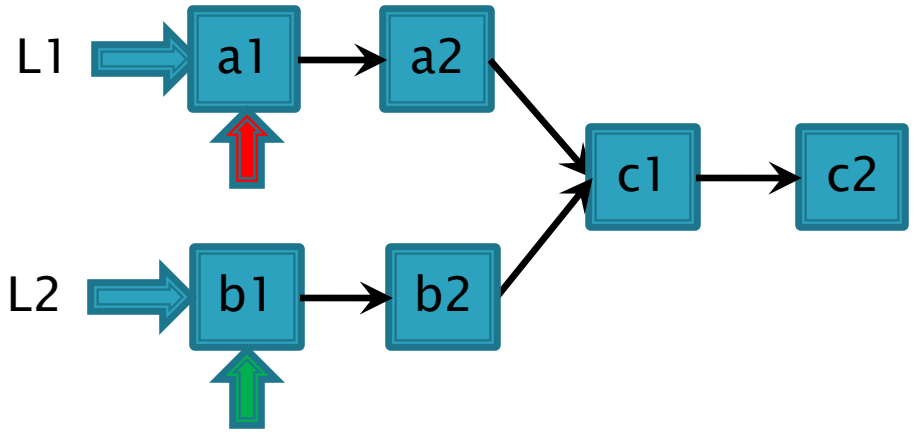
- Use pointer A to traverse L1, and use pointer B to traverse L2
- Compare every possible pair of A and B

Method #1

```
A = L1.head
while A != NULL
        B = L2.head
        while B != NULL
                if A == B
                        return A
                B = B.next
        A = A.next
```

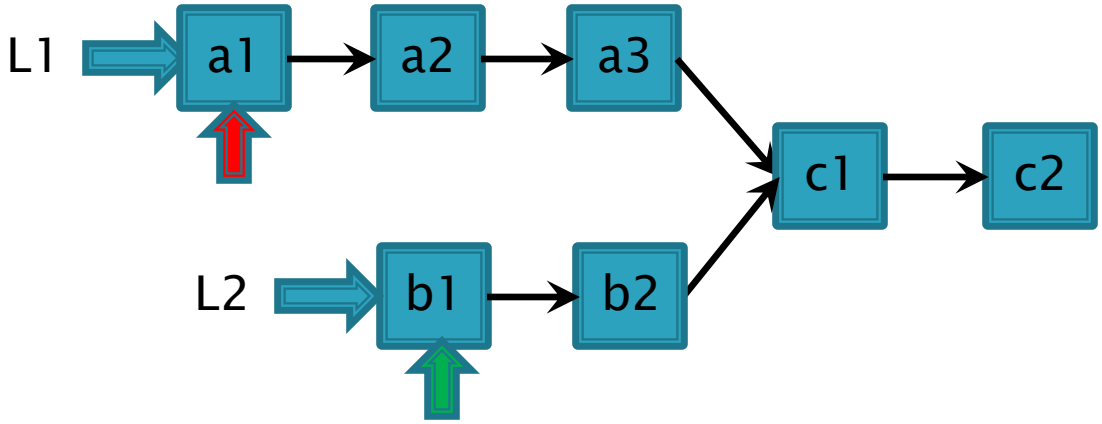
O(MN)

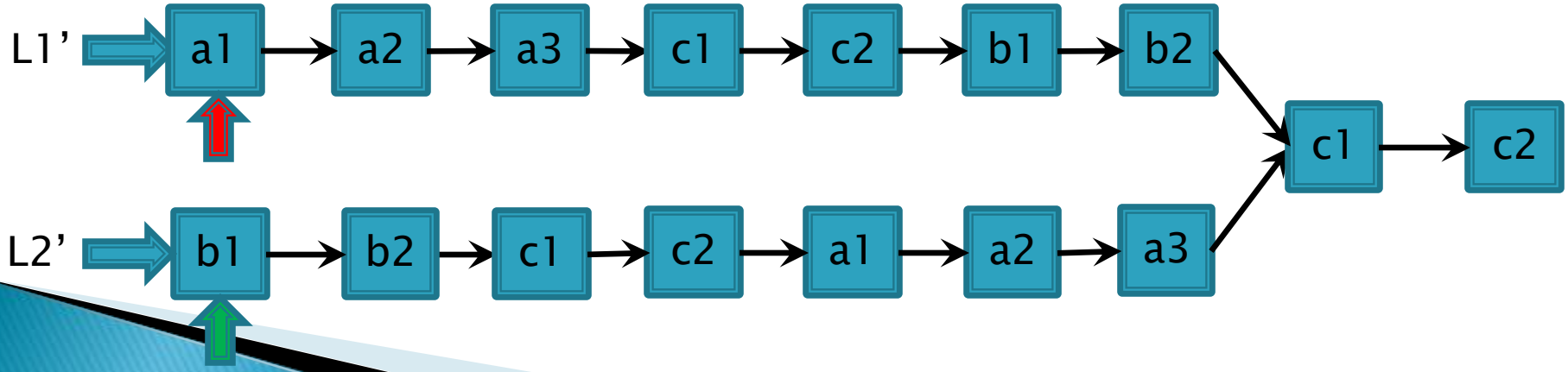
Consider a special case



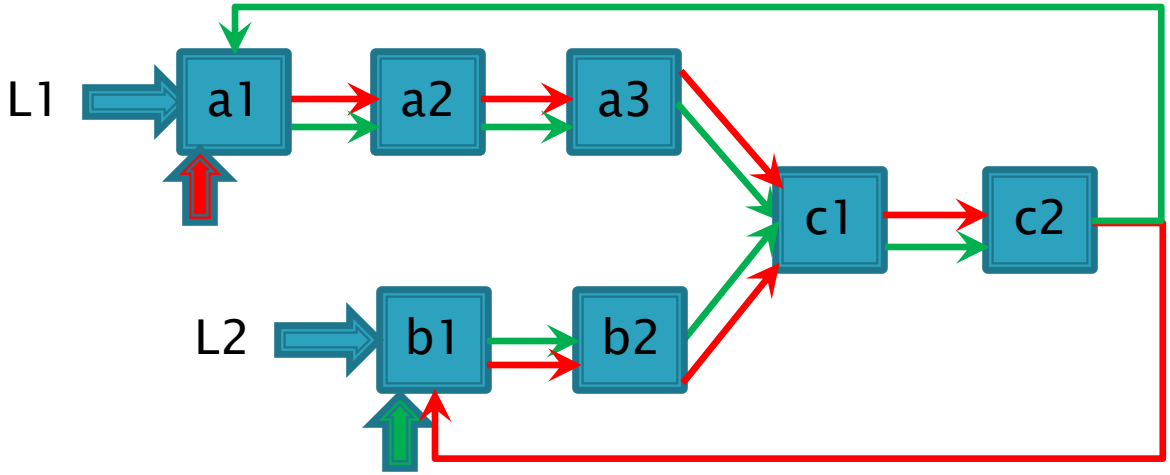
If L1 and L2 have the same length, the problem is easy to solve

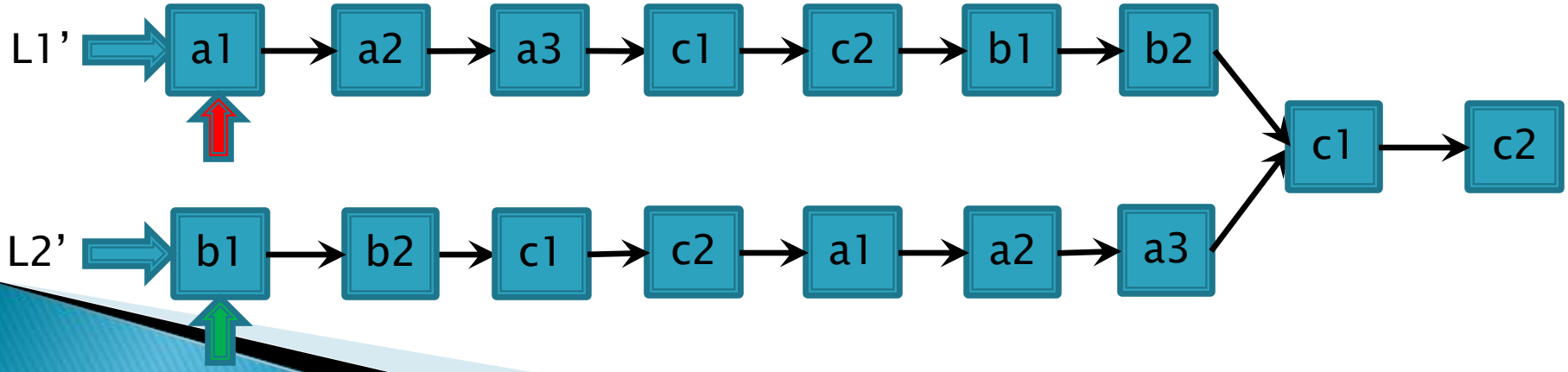
Method #2


L1
a1
a2
a3
c1
c2
L2
b1
b2


L1'
a1
a2
a3
c1
c2
b1
b2
L2'
b1
b2
c1
c2
a1
a2
a3
c1
c2

Method #2


L1
a1
a2
a3
c1
c2
L2
b1
b2


L1'
a1
a2
a3
c1
c2
b1
b2
L2'
b1
b2
c1
c2
a1
a2
a3
c1
c2

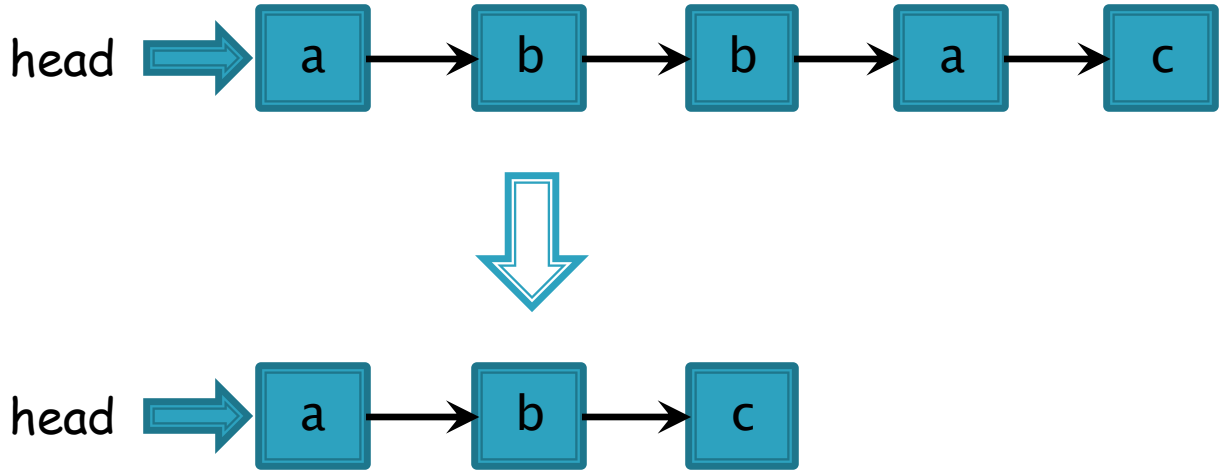
Method #2

```
A = L1.head
B = L2.head
while TRUE
        if A == B
                return A
        if A.next == NULL
                A = L2.head
        else A = A.next
        if B.next == NULL
                B = L1.head
        else B = B.next
```

O(M+N)

Exercise 2: duplicate letter detection

Given the head of a singly linked list *L*, in which each node's data is a lowercase letter, remove all the nodes with duplicate lowercase letters



Method #1

```
If L.head = null return
A = L.head.next
while A != null
    dataA = A.data
    B = A.next, pre = A
    while B.next != null
        dataB = B.data
        if dataB == dataA
            pre.next = B.next
        pre = B
        B = B.next
    A=A.next
```

$O(M^2)$

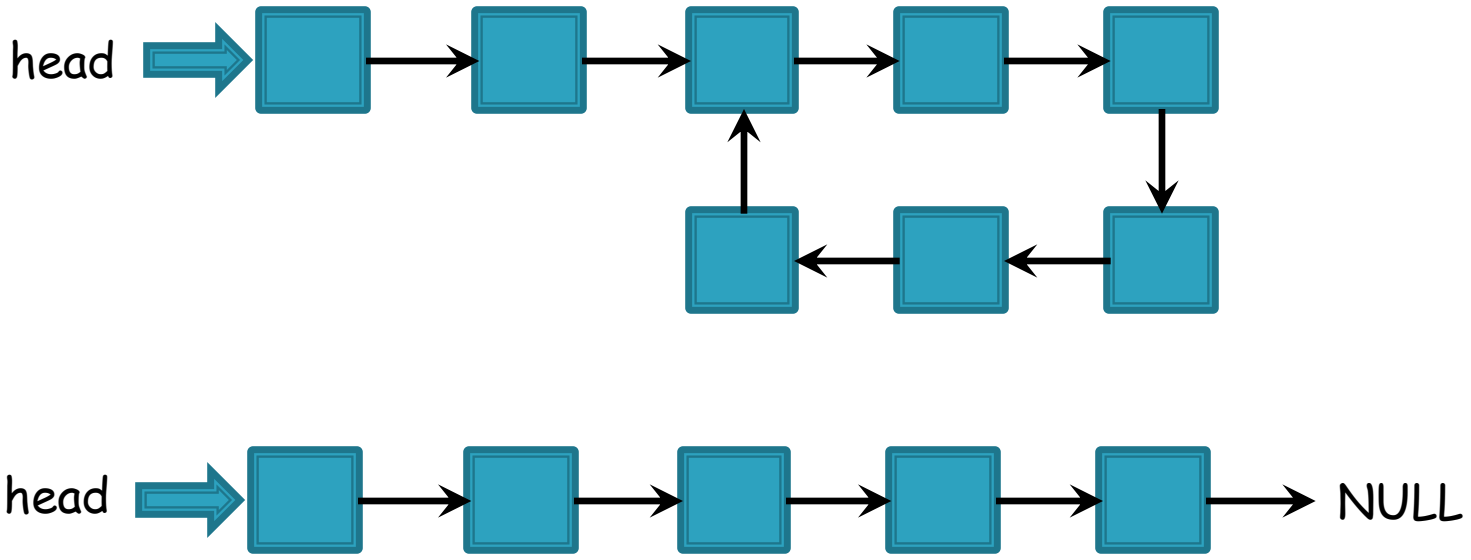
Method #2

```
Boolean b[ ] = new Boolean[26]
A = L.head, pre = L.head
while A.next != null
    dataA = A.data
    index = dataA – 'a'
    if b[index] == false
        b[index] = true
        pre = A
    else
        pre.next = A.next
A = A.next
```

O(M)

Exercise 3: cycle detection

Given the head of a singly linked list *L*, decide if *L* has a cycle



Method #1

- If *L* is acyclic, we ultimately arrive at NULL by continuously following the next pointer:

```
p = L.head
for i = 1 upto M
        if p == NULL
                return "acyclic"
        else   p = p.next
return "cyclic"
```

M is some big number

- M must be sufficiently large to guarantee correctness, but it is hard to decide M

Method #2

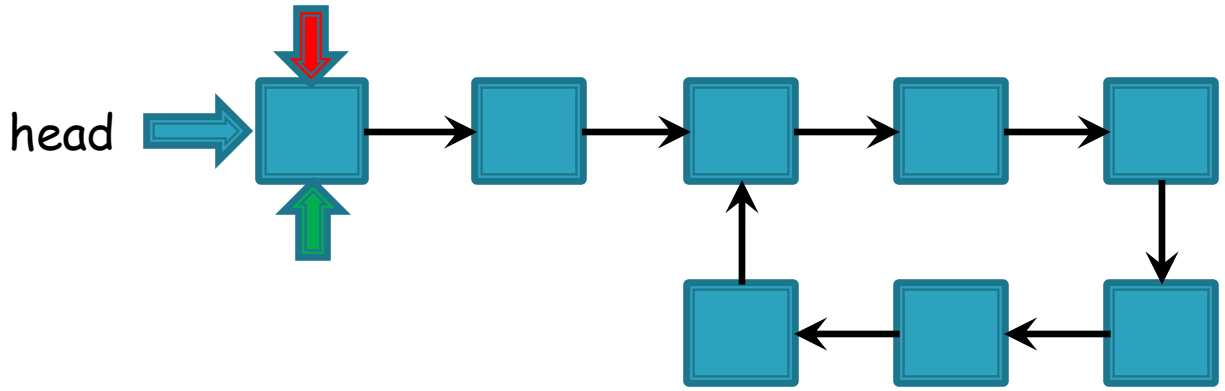


- Store all revisited noded in a new list L':

```
p = L.head
while p != NULL
        if search(L', p) == NULL
                insert(L',p)
                p = p.next
        else return "cyclic"
return "acyclic"
```

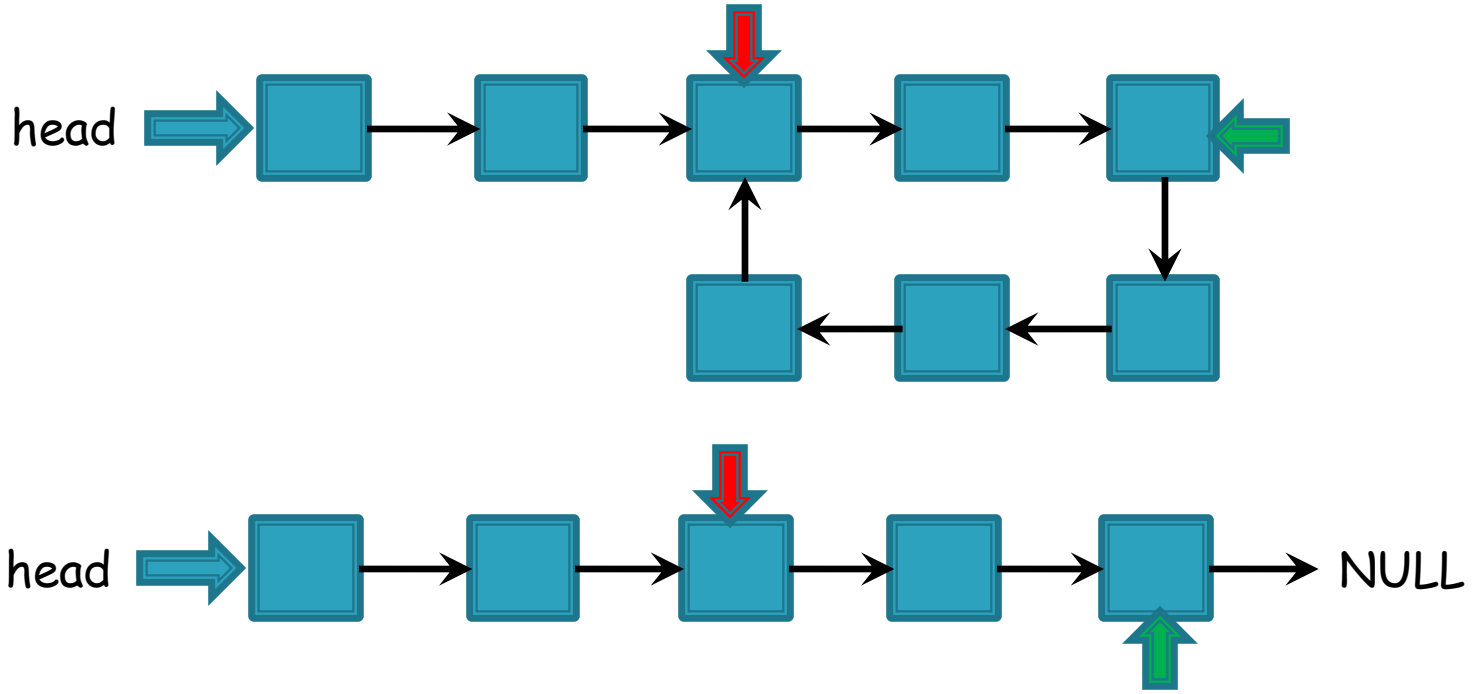
- Search on L' is expensive; use a Hash table

Method #3



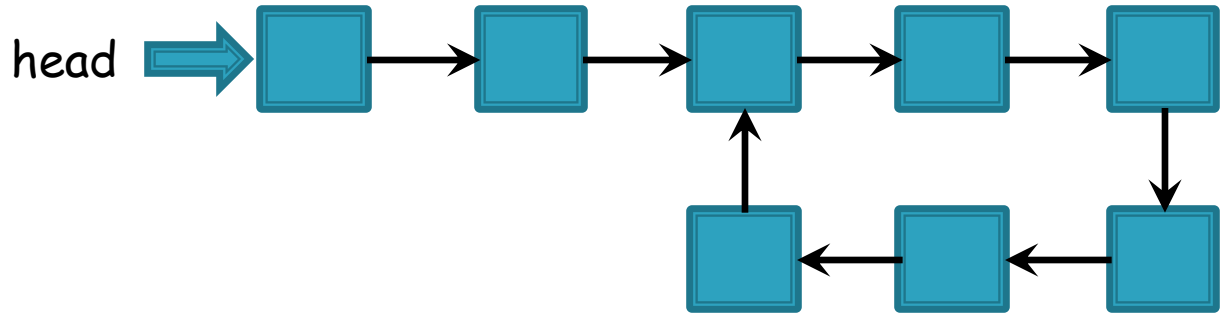
- Use two pointers A and B, both initialized to head
- Every time A=A.next while B=B.next.next

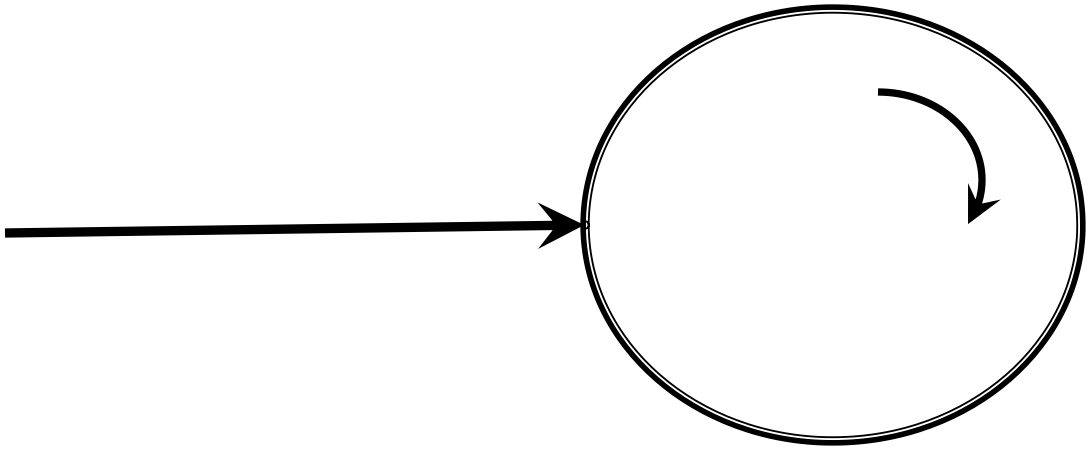
Method #3



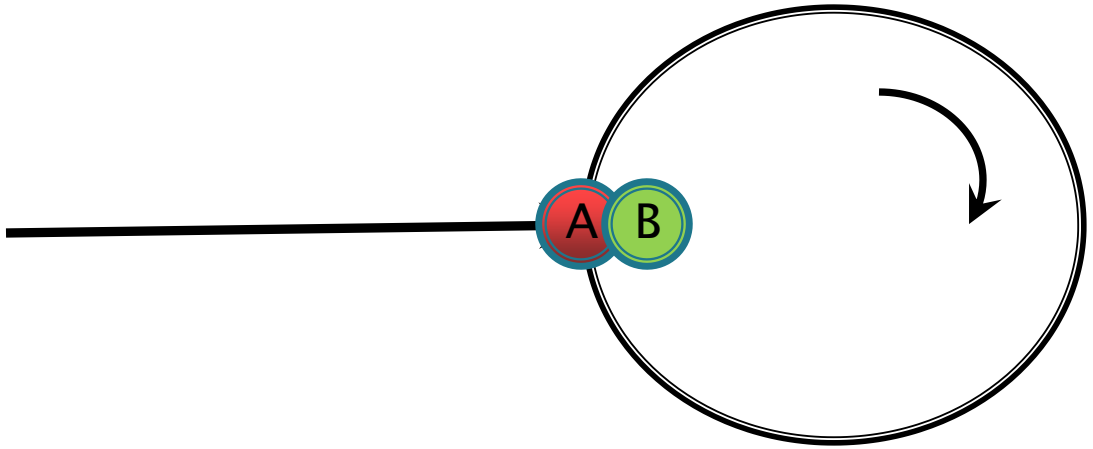
- If *L* is acyclic, either B or B.next be NULL
- If *L* is cyclic, B enters the cycle earlier than *A*

Method #3



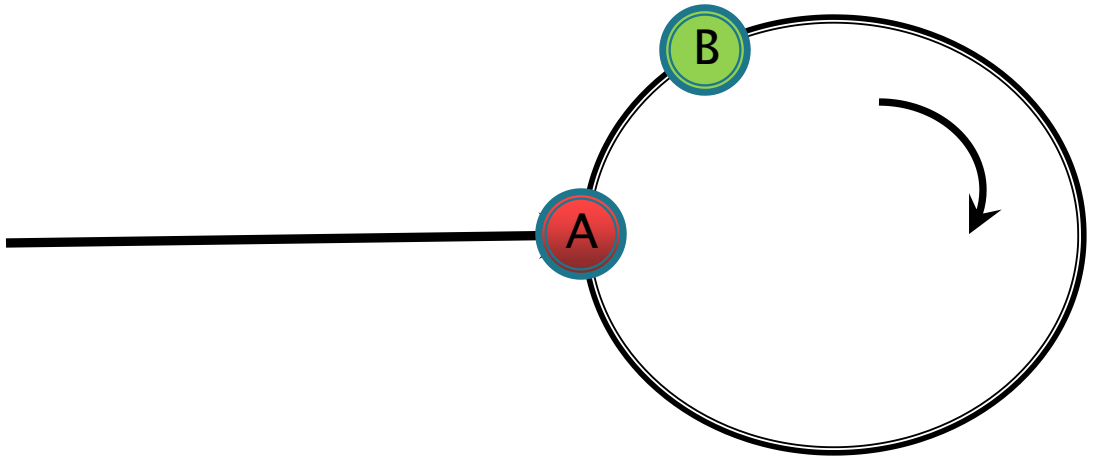


Method #3



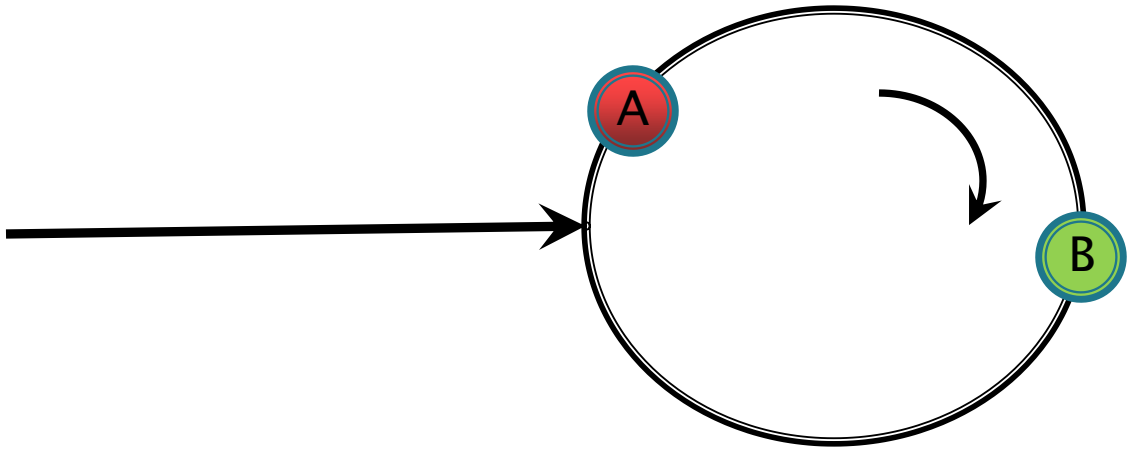
- Case I: B is exactly at entrance when A arrives at the cycle
- So A and B meet at entrance

Method #3



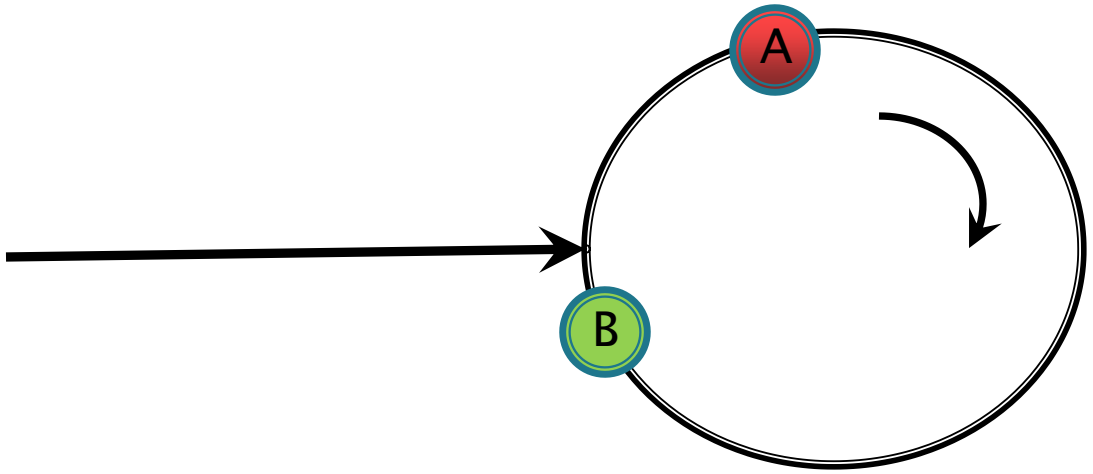
- Case II: B is somewhere else in the cycle when A arrives at entrance

Method #3



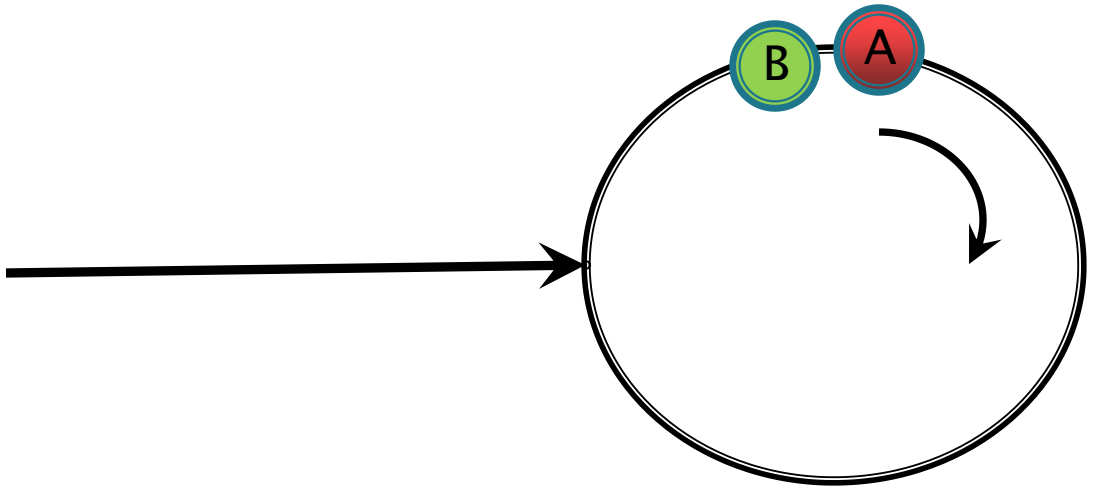
- Since B is moving faster, it must overtake A at a certain point in time
- After t timestamps, the distance gap is (2-1)t = t
- The distance of a circle is x, which is a constant
- So the distances between A and B are 0,1,2,…, t-1

Method #3



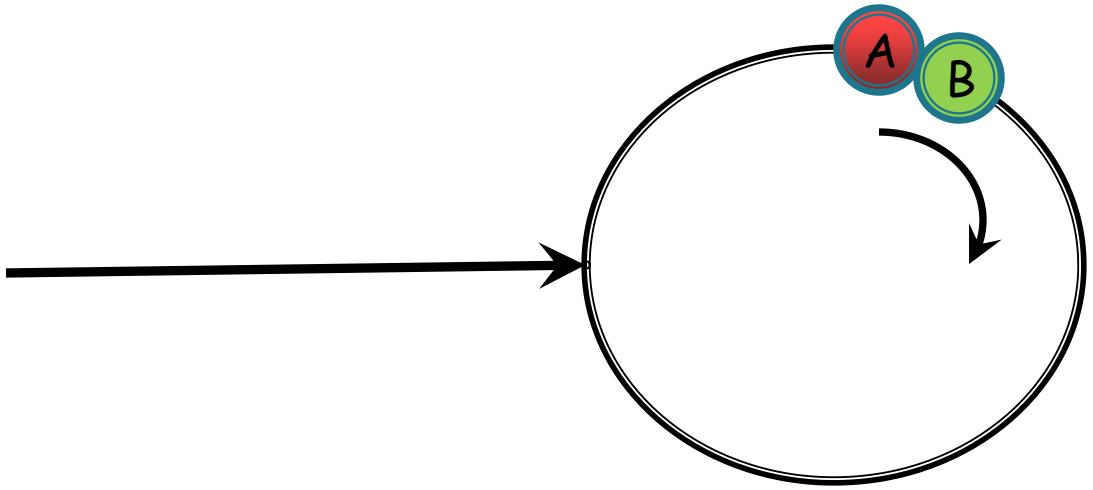
- Since B is moving faster, it must overtake A at a certain point in time

Method #3



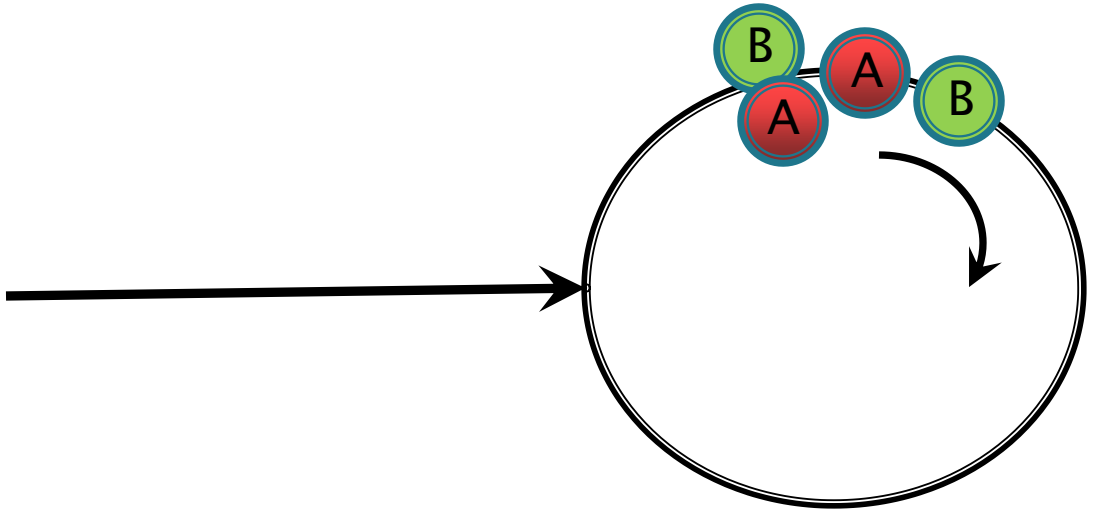
- Since B is moving faster, it must overtake A at a certain point in time

Method #3



- Since B is moving faster, it must overtake A at a certain point in time

Method #3



- And right before B overtakes A, the two nodes meet

Method #3

- Thus, A and B are guaranteed to meet if list is cyclic

```
A = L.head; B = L.head
while B != NULL and B.next != NULL
        if A == B
                return "cyclic"
        A = A.next
        B = B.next.next
return "acyclic"
```